```
logger = ActiveSupport::TaggedLogging.new(Logger.new(STDOUT))
logger.tagged("BCX") { logger.info "Stuff"
}                            # Logs "[BCX] Stuff"
logger.tagged("BCX", "Jason") { logger.info "Stuff"
}                   # Logs "[BCX] [Jason] Stuff"
logger.tagged("BCX") { logger.tagged("Jason") { logger.info "Stuff" } }
# Logs "[BCX] [Jason] Stuff"
```

2.5 Impact of Logs on Performance

Logging will always have a small impact on performance of your rails app, particularly when logging to disk. However, there are a few subtleties:

Using the `:debug` level will have a greater performance penalty than `:fatal`, as a far greater number of strings are being evaluated and written to the log output (e.g. disk).
Another potential pitfall is that if you have many calls to `Logger` like this in your code:

```
logger.debug "Person attributes hash: #{@person.attributes.inspect}"
```

In the above example, There will be a performance impact even if the allowed output level doesn't include debug. The reason is that Ruby has to evaluate these strings, which includes instantiating the somewhat heavy `String` object and interpolating the variables, and which takes time. Therefore, it's recommended to pass blocks to the logger methods, as these are only evaluated if the output level is the same or included in the allowed level (i.e. lazy loading). The same code rewritten would be:

```
logger.debug {"Person attributes hash: #{@person.attributes.inspect}"}
```

The contents of the block, and therefore the string interpolation, is only evaluated if debug is enabled. This performance savings is only really noticeable with large amounts of logging, but it's a good practice to employ.

3 Debugging with the `web-console` gem

The web console allows you to start an interactive Ruby session in your browser. An interactive console is launched automatically in case of an error but can also be launched for debugging purposes by invoking `console` in a view or controller.
For example in a view:

```
# new.html.erb
<%= console %>
```

Or in a controller:

```
# posts_controller.rb
class PostsController < ApplicationController
  def new
    console
    @post = Post.new
  end
end
```

3.1 config.web_console.whitelisted_ips

By default the web console can only be accessed from localhost. `config.web_console.whitelisted_ips` lets you control which IPs have access to the console.
For example, to allow access from both localhost and 192.168.0.100, you can put inside your configuration file: